

COURSE CODE : CSA1304

COURSE NAME : THEORY OF COMPUTATION

Girija B

192311044

Lab Exercise

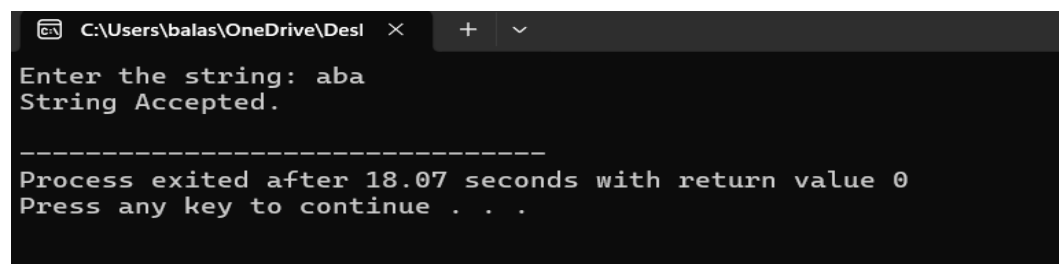
1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

CODE:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // Check if the string starts with 'a' and ends with 'a'
    if (str[0] == 'a' && str[len - 1] == 'a') {
        // Check whether all characters are either 'a' or 'b'
        for (i = 0; i < len; i++) {
            if (str[i] != 'a' && str[i] != 'b') {
                printf("Invalid input! Use only 'a' and 'b'.\n");
                return 0;
            }
        }
        printf("String Accepted.\n");
    } else {
        printf("String Rejected.\n");
    }

    return 0;
}
```

OUTPUT:



```
C:\Users\balas\OneDrive\Desl >
Enter the string: aba
String Accepted.

-----
Process exited after 18.07 seconds with return value 0
Press any key to continue . . .
```

2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

CODE:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len;
    printf("Enter the binary string: ");
```

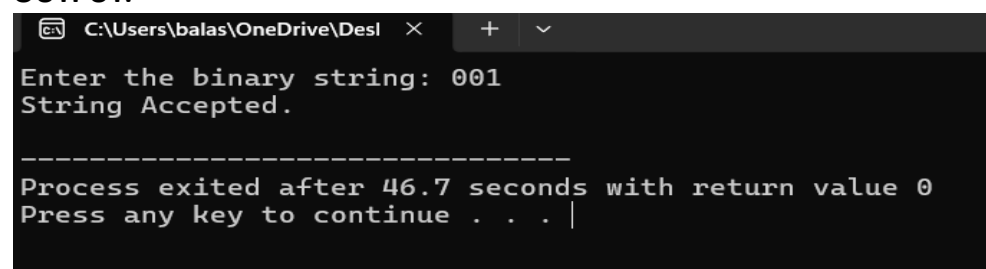
```

scanf("%s", str);
len = strlen(str);
// Check if the string starts with '0' and ends with '1'
if (str[0] == '0' && str[len - 1] == '1') {
    // Check whether all characters are either '0' or '1'
    for (i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') {
            printf("Invalid input! Use only '0' and '1'.\n");
            return 0;
        }
    }
    printf("String Accepted.\n");
} else {
    printf("String Rejected.\n");
}

return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desktop >
Enter the binary string: 001
String Accepted.

-----
Process exited after 46.7 seconds with return value 0
Press any key to continue . . . |

```

3. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) $S \rightarrow 0A1$ $A \rightarrow 0A \mid 1A \mid \epsilon$

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // String must have at least 2 characters (0 and 1)
    if (len < 2) {
        printf("String Rejected.\n");
        return 0;
    }
    // Check if string starts with 0 and ends with 1
    if (str[0] != '0' || str[len - 1] != '1') {

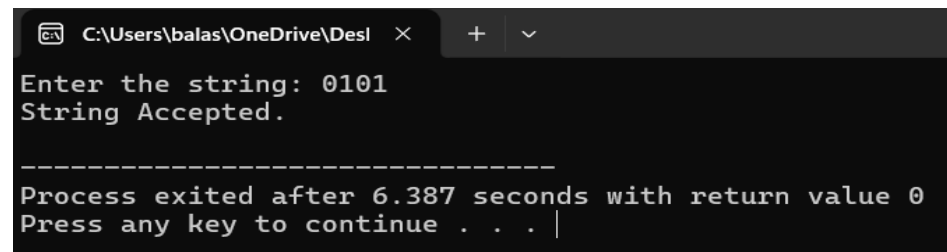
```

```

    printf("String Rejected.\n");
    return 0;
}
// Check if all characters are either 0 or 1
for (i = 0; i < len; i++) {
    if (str[i] != '0' && str[i] != '1') {
        printf("Invalid input! Use only 0 and 1.\n");
        return 0;
    }
}
printf("String Accepted.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desl  ×  +  ▾
Enter the string: 0101
String Accepted.

-----
Process exited after 6.387 seconds with return value 0
Press any key to continue . . . |

```

4. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len, flag = 1;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // Check whether all characters are 0 or 1
    for(i = 0; i < len; i++) {
        if(str[i] != '0' && str[i] != '1') {
            printf("Invalid input! Use only 0 and 1.\n");
            return 0;
        }
    }
    // Check palindrome
    for(i = 0; i < len / 2; i++) {
        if(str[i] != str[len - 1 - i]) {
            flag = 0;
            break;
        }
    }
}

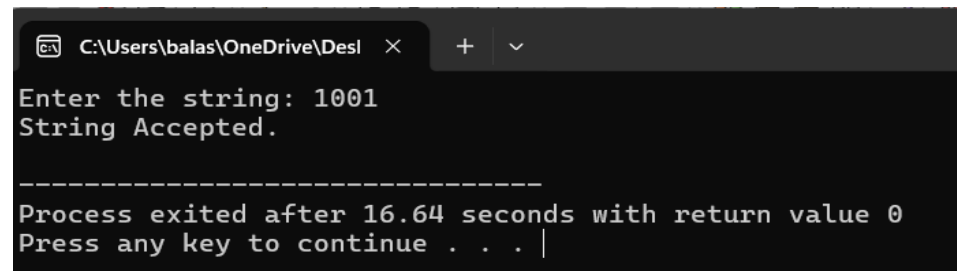
```

```

}
if(flag)
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desl >
Enter the string: 1001
String Accepted.

-----
Process exited after 16.64 seconds with return value 0
Press any key to continue . . . |

```

5. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) $S \rightarrow 0S0 \mid A A \rightarrow 1A \mid \epsilon$

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len, i = 0, leftZeros = 0, rightZeros = 0;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // Check whether all characters are 0 or 1
    for (i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') {
            printf("Invalid input! Use only 0 and 1.\n");
            return 0;
        }
    }
    // Count leading zeros
    i = 0;
    while (i < len && str[i] == '0') {
        leftZeros++;
        i++;
    }
    // Skip middle ones
    while (i < len && str[i] == '1') {
        i++;
    }
}

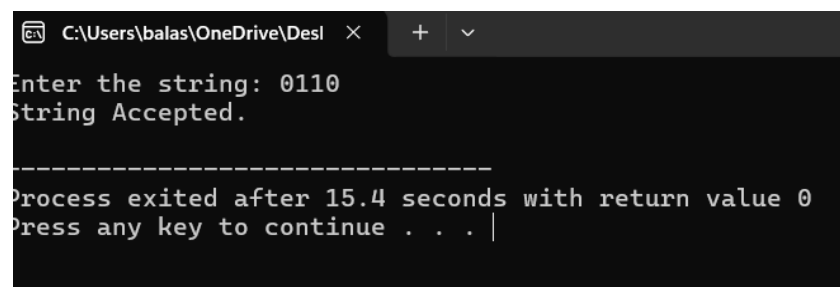
```

```

// Count trailing zeros
while (i < len && str[i] == '0') {
    rightZeros++;
    i++;
}
// Extra characters after trailing zeros
if (i != len) {
    printf("String Rejected.\n");
    return 0;
}
// Check equal leading and trailing zeros
if (leftZeros == rightZeros)
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desl > Enter the string: 0110
String Accepted.

-----
Process exited after 15.4 seconds with return value 0
Press any key to continue . . . |

```

- Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) $S \rightarrow 0S1 \mid \epsilon$

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len, i = 0;
    int zeros = 0, ones = 0;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // Check if all characters are 0 or 1
    for (i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') {
            printf("Invalid input! Use only 0 and 1.\n");
            return 0;
        }
    }
}

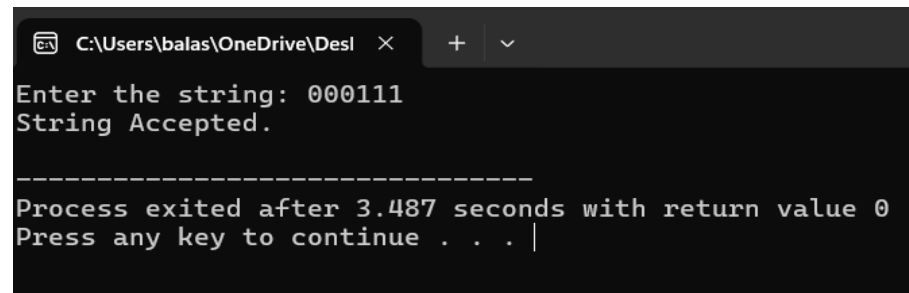
```

```

i = 0;
// Count leading 0's
while (i < len && str[i] == '0') {
    zeros++;
    i++;
}
// Count trailing 1's
while (i < len && str[i] == '1') {
    ones++;
    i++;
}
// Reject if extra characters exist
if (i != len) {
    printf("String Rejected.\n");
    return 0;
}
// Check equal number of 0's and 1's
if (zeros == ones)
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desl >
Enter the string: 000111
String Accepted.

-----
Process exited after 3.487 seconds with return value 0
Press any key to continue . . .

```

7. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) $S \rightarrow A101A$, $A \rightarrow 0A \mid 1A \mid \epsilon$

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len, found = 0;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);

```

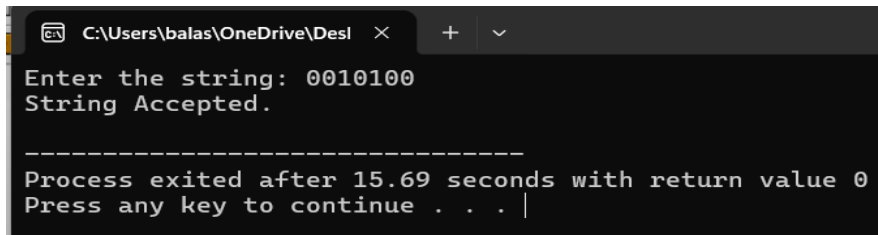
```

// Check whether all characters are 0 or 1
for (i = 0; i < len; i++) {
    if (str[i] != '0' && str[i] != '1') {
        printf("Invalid input! Use only 0 and 1.\n");
        return 0;
    }
}
// Check for substring "101"
for (i = 0; i <= len - 3; i++) {
    if (str[i] == '1' && str[i + 1] == '0' && str[i + 2] == '1') {
        found = 1;
        break;
    }
}
if (found)
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");

return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desktop >
Enter the string: 0010100
String Accepted.
-----
Process exited after 15.69 seconds with return value 0
Press any key to continue . . . |

```

8. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    // String should not be empty
    if (len == 0) {
        printf("String Rejected.\n");
        return 0;
    }
}

```

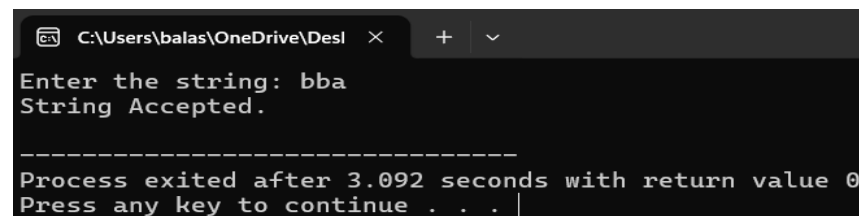


```

// Check whether all characters are either 'a' or 'b'
for (i = 0; i < len; i++) {
    if (str[i] != 'a' && str[i] != 'b') {
        printf("Invalid input! Use only 'a' and 'b'.\n");
        return 0;
    }
}
// Check if string starts with 'b' and ends with 'a'
if (str[0] == 'b' && str[len - 1] == 'a')
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desktop >
Enter the string: bba
String Accepted.
-----
Process exited after 3.092 seconds with return value 0
Press any key to continue . . .

```

9. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len;
    printf("Enter the binary string: ");
    scanf("%s", str);
    len = strlen(str);
    // Check for empty string
    if (len == 0) {
        printf("String Rejected.\n");
        return 0;
    }
    // Check whether all characters are 0 or 1
    for (i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') {
            printf("Invalid input! Use only 0 and 1.\n");
            return 0;
        }
    }
}

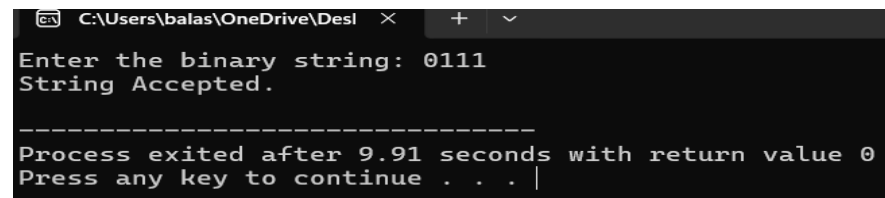
```

```

// Check if string starts with 0 and ends with 1
if (str[0] == '0' && str[len - 1] == '1')
    printf("String Accepted.\n");
else
    printf("String Rejected.\n");
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desktop >
Enter the binary string: 0111
String Accepted.
-----
Process exited after 9.91 seconds with return value 0
Press any key to continue . . . |

```

10. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

CODE:

```

#include <stdio.h>
int n, e[10][10], visited[10];
// Function to find epsilon closure
void epsilonClosure(int state)
{
    int i;
    visited[state] = 1;
    printf("q%d ", state);
    for(i = 0; i < n; i++)
    {
        if(e[state][i] == 1 && !visited[i])
        {
            epsilonClosure(i);
        }
    }
}
int main()
{
    int i, j;
    printf("Enter the number of states: ");
    scanf("%d", &n);

    printf("Enter the epsilon transition matrix (0 or 1):\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &e[i][j]);
        }
    }
}

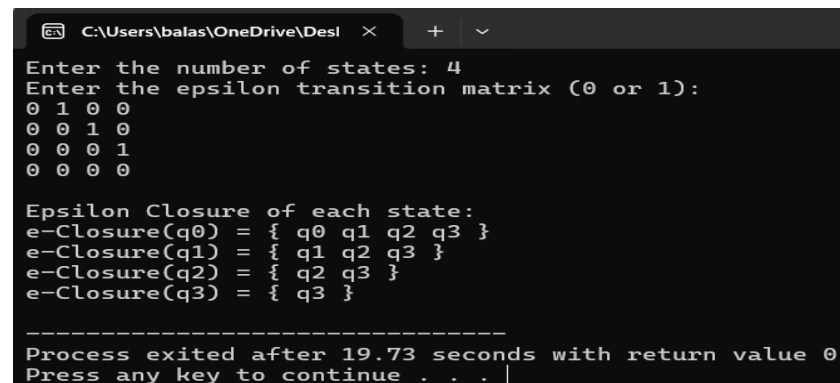
```

```

    }
}
printf("\nEpsilon Closure of each state:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
        visited[j] = 0;
    printf("ε-Closure(q%d) = { ", i);
    epsilonClosure(i);
    printf("}\n");
}
return 0;
}

```

OUTPUT:



```

C:\Users\balas\OneDrive\Desl  ×  +  ▾
Enter the number of states: 4
Enter the epsilon transition matrix (0 or 1):
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0

Epsilon Closure of each state:
e-Closure(q0) = { q0 q1 q2 q3 }
e-Closure(q1) = { q1 q2 q3 }
e-Closure(q2) = { q2 q3 }
e-Closure(q3) = { q3 }

-----
Process exited after 19.73 seconds with return value 0
Press any key to continue . . .

```

11. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

CODE:

```

#include <stdio.h>
int n, e[10][10], visited[10];
// Function to find epsilon closure
void epsilonClosure(int state)
{
    int i;
    visited[state] = 1;
    printf("q%d ", state);
    for(i = 0; i < n; i++)
    {
        if(e[state][i] == 1 && !visited[i])
        {
            epsilonClosure(i);
        }
    }
}
int main()
{

```

```

int i, j;
printf("Enter the number of states: ");
scanf("%d", &n);
printf("Enter the epsilon transition matrix (0 or 1):\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        scanf("%d", &e[i][j]);
    }
}
printf("\nEpsilon Closure of each state:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
        visited[j] = 0;
    printf("ε-Closure(q%d) = { ", i);
    epsilonClosure(i);
    printf("}\n");
}
return 0;
}

```

OUTPUT:

```

C:\Users\balas\OneDrive\Desl > Enter the number of states: 4
Enter the epsilon transition matrix (0 or 1):
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0

Epsilon Closure of each state:
e-Closure(q0) = { q0 q1 q2 q3 }
e-Closure(q1) = { q1 q2 q3 }
e-Closure(q2) = { q2 q3 }
e-Closure(q3) = { q3 }

-----
Process exited after 21.36 seconds with return value 0
Press any key to continue . . . |

```